Анализ данных с использованием языка Python

Пояснения к коду. Семинар 8

Matplotlib и Seaborn: базовые графики и оформление

А. Н. Лата

Пояснения к ноутбуку `notebooks/seminar-08-matplotlib-seaborn.ipynb`. Номер ячейки — её порядковый номер среди ячеек кода ноутбука, считая сверху. Для каждой ячейки приведён её код с номерами строк, а под ним — что делает каждая команда и что означает каждый аргумент именно в этой ячейке. Код совпадает с ноутбуком, только пробелы перед комментариями сокращены. Полные подписи функций со всеми параметрами — в справочнике `reference/visualization.md`.

1 Зачем график и как он устроен

Ячейка 1. Подключение библиотек и загрузка таблицы ДТП

```
1  import os
2  import matplotlib.pyplot as plt
3  import numpy as np
4  import pandas as pd
5  import seaborn as sns
6
7  sns.set_theme(style='whitegrid')  # оформление графиков по умолчанию
8  os.makedirs('img', exist_ok=True)  # папка для картинок
9  dtp = pd.read_csv('../datasets/legal/dtp-bryansk.csv', index_col='id', ↩
   ↪ parse_dates=['datetime'])  # ДТП Брянской области
10 dtp.shape
```

– Строки 1–5. `import` подключает библиотеку, `as` даёт ей короткое имя: дальше в коде пишем `plt`, `np`, `pd`, `sns`. Новых библиотек три: `os` — работа с папками и файлами на компьютере; `matplotlib.pyplot` — та часть библиотеки matplotlib, которая создаёт рисунки, точка в имени означает «модуль pyplot внутри matplotlib»; `seaborn` — статистические графики из таблицы.

– `sns.set_theme(style='whitegrid')` — один раз задаёт оформление всех графиков ноутбука. Аргумент `style='whitegrid'` — белый фон с бледной сеткой. Оформление действует и на графики, которые рисуют pandas и matplotlib, поэтому строка стоит в первой ячейке.

– `os.makedirs('img', exist_ok=True)` — создаёт папку `img` рядом с ноутбуком; в неё в конце семинара сохраняется картинка. Аргумент `exist_ok=True` — если папка уже есть, это не ошибка: ячейку можно запускать повторно.

– `pd.read_csv(...)` — читает таблицу из файла CSV. Первый аргумент — путь к файлу, `'../datasets/legal/dtp-bryansk.csv'`; две точки в его начале означают: подняться из папки `notebooks` на уровень выше и оттуда спуститься в `datasets/legal`. Аргумент `index_col='id'` — столбец `id` становится индексом: номер ДТП служит именем строки. Аргумент `parse_dates=['datetime']` — столбец `datetime` читается как дата и время, а не как текст; без этого не работали бы `dt.year` и другие свойства дат.

– `dtp = ...` — сохраняет таблицу под именем `dtp`.

– `dtp.shape` — размер таблицы: число строк и число столбцов. Печатается `(10114, 14)`; индекс `id` в число столбцов не входит.

Ячейка 2. Сводка: число ДТП по годам

```
by_year = dtp.groupby(dtp['datetime'].dt.year).size()  # ДТП по годам
by_year
```

Первую строку удобно читать изнутри наружу.

- `dtp['datetime']` — столбец с датой и временем каждого ДТП.
- `.dt.year` — год каждой даты: Series из 10 114 чисел вроде 2015, по одному на ДТП.
- `dtp.groupby(...)` — делит строки таблицы на группы по этим годам. В скобках стоит не имя столбца, а сами годы: группировать можно по любому Series той же длины, что и таблица.
- `.size()` — число строк в каждой группе, то есть число ДТП в каждом году.
- `by_year = ...` — сохраняет сводку под именем `by_year`: она понадобится в нескольких следующих ячейках.
- Результат — Series: в индексе годы, в значениях число ДТП. Индекс называется `datetime`, по имени столбца, из которого взят год; поэтому в следующей ячейке ось X будет подписана этим словом.
- Вторая строка, `by_year`, — просто имя: Jupyter печатает значение последней строки ячейки, и под ячейкой появляется сводка.

Ячейка 3. Фигура, оси и линия по годам

```
fig, ax = plt.subplots(figsize=(8, 4))  # фигура и оси
by_year.plot(ax=ax, marker='o')  # линия по годам
```

- `plt.subplots(figsize=(8, 4))` — создаёт фигуру, то есть лист, и на нём одни оси, то есть рамку для графика. Аргумент `figsize=(8, 4)` — размер листа в дюймах: 8 в ширину, 4 в высоту.
- `fig, ax = ...` — функция возвращает два объекта сразу, и они раскладываются по двум именам по порядку: первый, фигура, попадает в `fig`, второй, оси, — в `ax`.
- `by_year.plot(...)` — метод `plot` у Series рисует линию: по оси X индекс, то есть годы, по оси Y значения, то есть число ДТП.
- `ax=ax` — рисовать в нашей рамке. Слева от знака равенства имя параметра, справа наша переменная; совпадение имён случайное, переменную можно было назвать иначе.
- `marker='o'` — кружок в каждой точке, чтобы были видны отдельные годы, а не только ломаная.
- Над графиком печатается `<Axes: xlabel='datetime'>`: это то, что вернул метод `plot`, сами оси. Jupyter печатает значение последней строки, а график выводит отдельно, под этой строкой.

Ячейка 4. Тот же график с подписями

```
fig, ax = plt.subplots(figsize=(8, 4))
by_year.plot(ax=ax, marker='o')
ax.set_title('ДТП в Брянской области по годам')  # заголовок
ax.set_xlabel('Год')  # подпись оси X
ax.set_ylabel('Число ДТП');  # подпись оси Y
```

– Строки 1–2 — те же, что в ячейке 3: новый лист и линия. График прошлой ячейки уже показан, и подписи, добавленные к нему в другой ячейке, на экране не появятся; поэтому все шаги одного графика пишутся в одной ячейке.

– `ax.set_title('...')` — заголовок над рамкой. Это метод осей: пишется через точку после `ax`, потому что заголовок принадлежит рамке.

– `ax.set_xlabel('Год')` и `ax.set_ylabel('Число ДТП')` — подписи оси X, горизонтальной, и оси Y, вертикальной.

– `;` в конце последней строки. Python точку с запятой в конце строки пропускает, а Jupyter понимает по ней: значение последней строки не печатать. Без неё над графиком появилась бы строка `Text(0, 0.5, 'Число ДТП')` — то, что вернул `set_ylabel`. На предыдущих строках точка с запятой не нужна: Jupyter печатает значение только последней строки.

2 Линия: время

Ячейка 5. ДТП по месяцам за все годы

```
by_month = dtp.groupby(dtp['datetime'].dt.month).size()  # ДТП по месяцам ↩
↪ за все годы
fig, ax = plt.subplots(figsize=(8, 4))
by_month.plot(ax=ax, marker='o')
ax.set_title('ДТП по месяцам, 2015-2025')
ax.set_xlabel('Месяц')
ax.set_ylabel('Число ДТП');
```

– `dt.month` — номер месяца каждой даты, от 1 до 12. Остальное в первой строке — как в ячейке 2: группы по месяцам и число строк в каждой. Месяцы всех одиннадцати лет попадают в одну группу: в группе 8 все августы с 2015 по 2025 год.

– Строки 2–6 — скелет из ячейки 4: лист и рамка, линия, заголовок и подписи осей. Группировка сама ставит месяцы по порядку, поэтому линия идёт от января к декабрю.

Ячейка 6. Две линии в одной рамке

```
serious = dtp[dtp['severity'] != 'Легкий']  # ДТП без лёгких
fig, ax = plt.subplots(figsize=(8, 4))
by_year.plot(ax=ax, marker='o', label='Все ДТП')
serious.groupby(serious['datetime'].dt.year).size().plot(ax=ax, ↩
↪ marker='o', label='Без лёгких')
ax.legend()  # легенда из label
ax.set_title('ДТП по годам: все и без лёгких')
ax.set_xlabel('Год')
ax.set_ylabel('Число ДТП');
```

– `dtp['severity'] != 'Легкий'` — маска: `True` у ДТП, тяжесть которых не «Легкий»; знак `!=` означает «не равно». Значение пишется так, как в данных: «Легкий» через «е».

– `serious = dtp[...]` — строки, где маска `True`: таблица ДТП без лёгких.

– `by_year.plot(ax=ax, marker='o', label='Все ДТП')` — первая линия. Параметр `label` — подпись этой линии для легенды.

– Строка 4 — вторая линия одной строкой. Сначала сводка по годам для таблицы `serious`, как в ячейке 2: группировка `serious.groupby(serious['datetime'].dt.year)` и число строк `.size()`. Сразу за ней `.plot(...)` рисует эту сводку в ту же рамку со своей подписью. Методы, записанные подряд через точку, выполняются слева направо: каждый следующий работает с результатом предыдущего.

– `ax.legend()` — легенда: рамка с подписями линий в углу графика. Подписи она берёт из `label`; без этой строки подписи не видны. Цвета линий matplotlib выбирает сам: первая синяя, вторая оранжевая.

Ячейка 7. Линия по тексту: намеренная ошибка

```
1 dtp['category'].plot()  # линия по тексту: ошибка
```

– `dtp['category'].plot()` — линия по столбцу видов ДТП. pandas рисует только числа, а в столбце текст, поэтому `TypeError: no numeric data to plot`: «нет числовых данных для рисования». Ошибка намеренная: для категорий нужны столбцы.

3 Столбцы: категории

Ячейка 8. Шесть самых частых видов ДТП горизонтальными столбцами

```
1 top6 = dtp['category'].value_counts().head(6)  # шесть самых частых видов
2 fig, ax = plt.subplots(figsize=(8, 4))
3 top6.plot.barh(ax=ax)  # горизонтальные столбцы
4 ax.invert_yaxis()  # самый частый сверху
5 ax.set_title('Шесть самых частых видов ДТП')
6 ax.set_xlabel('Число ДТП')
7 ax.set_ylabel('');
```

– `dtp['category'].value_counts()` — сколько раз встречается каждый вид ДТП, по убыванию; `.head(6)` — первые шесть строк, то есть шесть самых частых видов. `top6` — Series: в индексе названия видов, в значениях число ДТП.

– `top6.plot.barh(ax=ax)` — горизонтальные столбцы, `barh` от «bar horizontal»: по столбцу на каждое значение, длина — число, подпись — из индекса.

– `ax.invert_yaxis()` — разворачивает вертикальную ось. pandas ставит первое значение сводки внизу, после разворота самый частый вид оказывается сверху.

– `ax.set_ylabel('')` — пустая подпись оси Y. Иначе там стояло бы `category`, имя индекса сводки, а названия видов и так подписаны у столбцов.

Ячейка 9. То же из сырой таблицы: seaborn считает сам

```
fig, ax = plt.subplots(figsize=(8, 4))
sns.countplot(data=dtp, y='category', order=top6.index, ax=ax)  # то же из ↩
↪ сырой таблицы
ax.set_title('Шесть самых частых видов ДТП')
ax.set_xlabel('Число ДТП')
ax.set_ylabel('');
```

– `sns.countplot(...)` — столбцы с числом строк для каждого значения столбца; считает сам, сводка не нужна.

– `data=dtp` — таблица, из которой брать столбцы; `y='category'` — имя столбца с категориями, по вертикальной оси, поэтому столбцы горизонтальные. С `x='category'` они были бы вертикальными.

– `order=top6.index` — какие значения рисовать и в каком порядке: названия шести видов из сводки ячейки 8, по убыванию. Остальные виды на рисунок не попадают. seaborn ставит первое значение сверху, разворачивать ось не нужно.

– `ax=ax` — рисовать в нашей рамке; у функций seaborn этот параметр называется так же, как у `plot`.

Ячейка 10. Столбцы средних: раненых на одно ДТП

```
frequent = dtp[dtp['category'].isin(top6.index)]  # ДТП шести частых видов
fig, ax = plt.subplots(figsize=(8, 4))
sns.barplot(data=frequent, x='injured_count', y='category', ↩
↪ order=top6.index, ax=ax)  # среднее число раненых
ax.set_title('Среднее число раненых в ДТП по видам')
ax.set_xlabel('Раненых на одно ДТП')
ax.set_ylabel('');
```

– `dtp['category'].isin(top6.index)` — маска: `True`, если вид ДТП входит в список шести частых; `isin` означает «входит в». `frequent` — ДТП только этих шести видов.

– `sns.barplot(...)` — столбцы средних: для каждой категории считает среднее числового столбца и рисует столбец такой длины.

– `x='injured_count'` — числовой столбец, среднее которого считается; `y='category'` — категории по вертикали; `order` — как в ячейке 9.

– Чёрные чёрточки на концах столбцов — усы, 95-процентный доверительный интервал среднего. seaborn рисует их сам; чем меньше ДТП в группе, тем они длиннее.

Ячейка 11. Тяжесть ДТП по годам: столбцы с накоплением

```
shares = pd.crosstab(dtp['datetime'].dt.year, dtp['severity'], ↩
↪ normalize='index')  # доли тяжести по годам
layers = shares[['Легкий', 'Тяжёлый', 'С погибшими']]  # слои снизу вверх
fig, ax = plt.subplots(figsize=(8, 4))
layers.plot(kind='bar', stacked=True, rot=0, ax=ax,
            color=['#c6dbef', '#6baed6', '#08519c'])  # три оттенка синего
ax.legend(title='Тяжесть')  # заголовок легенды
ax.set_title('Тяжесть ДТП по годам')
ax.set_xlabel('Год')
ax.set_ylabel('Доля ДТП');
```

– `pd.crosstab(dtp['datetime'].dt.year, dtp['severity'], normalize='index')` — таблица: строки — годы, столбцы — тяжесть, в клетках число ДТП. `normalize='index'` делит каждую строку на её сумму, и в клетках получаются доли внутри года.

– `shares[['Легкий', 'Тяжёлый', 'С погибшими']]` — три столбца в нужном порядке; внутренние квадратные скобки — список имён. Порядок в списке станет порядком слоёв снизу вверх. Столбец «Без пострадавших (нет данных)», в котором одно ДТП, в список не вошёл.

– `layers.plot(kind='bar', ...)` — вертикальные столбцы по строкам таблицы: на каждый год столбец, на каждый столбец таблицы свой цвет.

– `stacked=True` — ставить слои друг на друга, а не рядом; `rot=0` — подписи годов горизонтально, без поворота.

– Команда продолжается на строке 5: Python понимает, что команда не закончена, пока не закрыта скобка.

– `color=['#c6dbef', '#6baed6', '#08519c']` — цвета слоёв по порядку: светло-голубой, голубой, тёмно-синий. Цвет записан кодом: после решётки три пары знаков — сколько в цвете красного, зелёного и синего.

– `ax.legend(title='Тяжесть')` — легенда с русским заголовком. pandas подписал бы её именем столбца, `severity`.

4 Гистограмма и ящик с усами: распределение

Ячейка 12. Гистограмма часов: столбец на каждый час

```
hours = dtp['datetime'].dt.hour  # час каждого ДТП
fig, ax = plt.subplots(figsize=(8, 4))
sns.histplot(x=hours, discrete=True, ax=ax)  # столбец на каждый час
ax.set_title('ДТП по часам суток')
ax.set_xlabel('Час')
ax.set_ylabel('Число ДТП');
```

– `dtp['datetime'].dt.hour` — час каждого ДТП, число от 0 до 23. `hours` — отдельный Series, а не столбец таблицы.

– `sns.histplot(x=hours, ...)` — гистограмма; `x=hours` — значения передаются самим Series.

– `discrete=True` — значения целые: по столбцу на каждое целое число, и столбец стоит ровно над ним.

Ячейка 13. Те же часы, десять бинов

```
fig, ax = plt.subplots(figsize=(8, 4))
sns.histplot(x=hours, bins=10, ax=ax)  # те же часы, десять бинов
ax.set_title('ДТП по часам суток: десять бинов')
ax.set_xlabel('Час')
ax.set_ylabel('Число ДТП');
```

– `bins=10` — десять бинов — интервалов одинаковой ширины — от наименьшего значения до наибольшего: от 0 до 23, каждый шириной 2.3 часа. В один бин попадают то три целых часа, то два, и высоты столбцов из-за этого несравнимы. Остальное — как в ячейке 12.

Ячейка 14. Столбца нет в таблице: намеренная ошибка

```
sns.histplot(data=dtp, x='hour')  # столбца hour нет: ошибка
```

– `data=dtp, x='hour'` — другая форма вызова: таблица и имя столбца строкой. seaborn ищет в таблице столбец `hour`, не находит и сообщает: `ValueError: Could not interpret value` `` `hour` for `x`. An entry with this name does not appear in `data`. `` — «не удаётся понять значение hour для x: в data нет такого имени». Ошибка намеренная.

Ячейка 15. Час как столбец таблицы; раненых в одном ДТП

```
dtp['hour'] = hours  # час как столбец таблицы
fig, ax = plt.subplots(figsize=(8, 4))
sns.histplot(data=dtp, x='injured_count', discrete=True, ax=ax)  # раненых в ДТП
ax.set_title('Раненых в одном ДТП')
ax.set_xlabel('Раненых')
ax.set_ylabel('Число ДТП');
```

– `dtp['hour'] = hours` — новый столбец таблицы из Series `hours`: присваивание по новому имени добавляет столбец. Теперь форма `data=dtp, x='hour'` работает; столбец понадобится для тепловой карты в ячейке 18.

– `sns.histplot(data=dtp, x='injured_count', discrete=True, ax=ax)` — гистограмма числа раненых в одном ДТП, в форме «таблица и имя столбца»; раненые — целые числа, поэтому снова `discrete=True`.

Ячейка 16. Водители со своими ДТП

```
people = pd.read_csv('../datasets/legal/dtp-bryansk-participants.csv')  # участники ДТП
people['experience_clean'] = people['experience'].replace([95, 96, 99], np.nan)  # коды стажа пропусками
drivers = people[people['role'] == 'Водитель'].merge(dtp, left_on='dtp_id', right_index=True)  # водители с их ДТП
drivers.shape
```

– `pd.read_csv(...)` — таблица участников ДТП: строка — один участник. Индекс не задан, строки нумеруются с нуля.

– `people['experience'].replace([95, 96, 99], np.nan)` — в столбце стажа заменяет числа 95, 96 и 99 пропуском `np.nan`: это служебные коды, а не стаж. Результат записывается новым столбцом `experience_clean`, исходный остаётся.

– `people[people['role'] == 'Водитель']` — только водители.

– `.merge(dtp, left_on='dtp_id', right_index=True)` — присоединяет к каждому водителю строку его ДТП из `dtp`. `left_on='dtp_id'` — у водителей номер ДТП в столбце `dtp_id`; `right_index=True` — у `dtp` номер ДТП в индексе. Где номера совпали, строки склеены.

– `drivers.shape` — печатается `(15229, 27)`: 15 229 водителей, у каждого свои столбцы и столбцы его ДТП.

Ячейка 17. Стаж водителей по тяжести ДТП: ящики с усами

```
order = ['Легкий', 'Тяжёлый', 'С погибшими']  # порядок ящиков
fig, ax = plt.subplots(figsize=(8, 4))
sns.boxplot(data=drivers, x='severity', y='experience_clean', order=order, ↩
↪ ax=ax)  # ящики стажа по тяжести
ax.set_title('Стаж водителей по тяжести ДТП')
ax.set_xlabel('Тяжесть ДТП')
ax.set_ylabel('Стаж, лет');
```

– `order = [...]` — список групп в нужном порядке; значения пишутся так, как в данных, «Легкий» через «е».

– `sns.boxplot(...)` — ящики с усами по группам. `x='severity'` — группы по горизонтали; `y='experience_clean'` — числа, распределение которых сравнивается; `order` — какие группы рисовать и в каком порядке. Группа «Без пострадавших (нет данных)» из одного водителя в список не вошла и на рисунок не попала. Пропуски стажа seaborn пропускает сам.

5 Тепловая карта: две категории

Ячейка 18. ДТП по дням недели и часам

```
table = pd.crosstab(dtp['datetime'].dt.day_of_week.rename('День недели'), ↩
↪ dtp['hour'].rename('Час'))  # день недели × час
fig, ax = plt.subplots(figsize=(10, 4))
sns.heatmap(table, cmap='Blues', ax=ax)  # чем темнее, тем больше
ax.set_title('ДТП по дням недели и часам, понедельник 0');
```

– `dtp['datetime'].dt.day_of_week` — номер дня недели: 0 — понедельник, 6 — воскресенье.

– `.rename('День недели')` и `.rename('Час')` — меняют имя у Series. Тепловая карта берёт подписи осей из имён; без `rename` по вертикали стояло бы `datetime`, имя столбца, из которого взят день недели, а по горизонтали `hour`.

– `pd.crosstab(..., ...)` — таблица: строки — дни недели, столбцы — часы, в клетках число ДТП.

– `figsize=(10, 4)` — лист шире обычного: часов двадцать четыре.

– `sns.heatmap(table, cmap='Blues', ax=ax)` — тепловая карта: каждая клетка таблицы — цветной прямоугольник. `cmap='Blues'` — палитра от белого к тёмно-синему: чем больше число, тем темнее. Справа шкала: какому числу соответствует цвет.

Ячейка 19. Числа в клетках

```
small = pd.crosstab(dtp['datetime'].dt.day_of_week.rename('День недели'), ↩
↪ dtp['severity'].rename('Тяжесть'))  # день недели × тяжесть
fig, ax = plt.subplots(figsize=(8, 4))
sns.heatmap(small, cmap='Blues', annot=True, fmt='d', ax=ax)  # числа в ↩
↪ клетках
ax.set_title('Тяжесть ДТП по дням недели');
```

– Первая строка — такая же таблица, как в ячейке 18, но по дням недели и тяжести: 7 строк и 4 столбца. Столбцы тяжести идут по алфавиту.

– `annot=True` — вписать числа в клетки.

– `fmt='d'` — формат вписанных чисел: целые. Без него seaborn писал бы числа с двумя значащими цифрами, и 512 выглядело бы как `5.1e+02`.

6 Рассеяние как проверка данных

Ячейка 20. Координаты ДТП как записаны

```
fig, ax = plt.subplots(figsize=(8, 6))
ax.scatter(dtp['lon'], dtp['lat'], s=2, alpha=0.3)  # точки: долгота, широта
ax.set_title('Координаты ДТП: как записаны')
ax.set_xlabel('Долгота')
ax.set_ylabel('Широта');
```

– `figsize=(8, 6)` — лист почти квадратный: рисуем карту.

– `ax.scatter(dtp['lon'], dtp['lat'], ...)` — диаграмма рассеяния, метод самих осей matplotlib: по точке на каждую строку таблицы. Первый аргумент — положение по горизонтали, долгота; второй — по вертикали, широта.

– `s=2` — размер точки, маленький, чтобы соседние точки не сливались.

– `alpha=0.3` — прозрачность от 0, невидимо, до 1, непрозрачно. Где точки лежат друг на друге, цвет темнее, и густые места видны.

Ячейка 21. Только точки внутри рамки области

```
inside = dtp[dtp['lat'].between(51.8, 54.2) & dtp['lon'].between(31, ↩
↪ 35.5)]  # ДТП внутри рамки области
fig, ax = plt.subplots(figsize=(8, 6))
ax.scatter(inside['lon'], inside['lat'], s=2, alpha=0.3)
ax.set_title('Координаты ДТП внутри области')
ax.set_xlabel('Долгота')
ax.set_ylabel('Широта')
(dtp['lat'] == 0).sum(), len(dtp) - len(inside)  # с нулями, за рамкой
```

– `dtp['lat'].between(51.8, 54.2)` — маска: широта от 51.8 до 54.2, границы включаются.

– `&` — «и»: `True` там, где верны обе маски. Скобки вокруг масок здесь не нужны: `between` — вызов метода, а не сравнение.

– `inside = dtp[...]` — ДТП внутри рамки; строки 2–6 — та же диаграмма по ним.

– `(dtp['lat'] == 0).sum()` — сколько ДТП с нулевой широтой: каждое `True` считается единицей.

– `len(dtp) - len(inside)` — сколько ДТП за рамкой.

– Две величины через запятую Jupyter печатает в скобках: `(np.int64(224), 367)`. `np.int64(224)` — то же число 224, так выглядят числа numpy: сумму маски считает numpy, а `len` — обычный Python. В последней строке нет точки с запятой, поэтому числа напечатаны над картой.

7 Оформление и сохранение

Ячейка 22. Две панели: круговая диаграмма и столбцы

```
fig, axes = plt.subplots(1, 2, figsize=(12, 4))  # две панели в ряд
top6.plot.pie(ax=axes[0], ylabel='')  # круговая: антипример
top6.plot.barh(ax=axes[1])
axes[1].invert_yaxis()
axes[0].set_title('Круговая диаграмма')
axes[1].set_title('Столбцы')
axes[1].set_xlabel('Число ДТП')
axes[1].set_ylabel('')
fig.suptitle('Шесть видов ДТП: два способа показать')  # общий заголовок
fig.tight_layout();  # панели не наезжают
```

– `plt.subplots(1, 2, figsize=(12, 4))` — лист и на нём две рамки: одна строка, два столбца. Вместо одной рамки возвращается набор рамок `axes`: `axes[0]` — левая, `axes[1]` — правая, счёт с нуля.

– `top6.plot.pie(ax=axes[0], ylabel='')` — круговая диаграмма в левой рамке; `ylabel=''` — без подписи слева.

– Строки 3–8 — столбцы в правой рамке, как в ячейке 8, и подписи: каждая команда пишется после той рамки, к которой относится.

– `fig.suptitle(...)` — общий заголовок над всем листом; `set_title` даёт заголовок одной рамке.

– `fig.tight_layout()` — раздвигает рамки, чтобы подписи не налезали друг на друга.

Ячейка 23. Две панели по делу: месяцы и часы

```
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
by_month.plot(ax=axes[0], marker='o')
axes[0].set_title('По месяцам')
axes[0].set_xlabel('Месяц')
axes[0].set_ylabel('Число ДТП')
sns.histplot(x=hours, discrete=True, ax=axes[1])
axes[1].set_title('По часам')
axes[1].set_xlabel('Час')
axes[1].set_ylabel('Число ДТП')
fig.suptitle('Когда происходят ДТП в Брянской области')
fig.tight_layout();
```

– То же устройство, что в ячейке 22: слева линия по месяцам из ячейки 5, справа гистограмма часов из ячейки 12. Функция seaborn получает правую рамку так же, через `ax=axes[1]`.

– Фигура этой ячейки остаётся в переменной `fig`: её сохраняет ячейка 25.

Ячейка 24. Сохранение без имени фигуры: ловушка

```
plt.savefig('img/wrong.png')  # текущей фигуры уже нет
os.path.getsize('img/wrong.png')  # размер файла в байтах
```

– `plt.savefig('img/wrong.png')` — сохраняет в файл «текущую» фигуру. Показав график под прошлой ячейкой, Jupyter закрыл его фигуру, текущей нет, и команда создала и сохранила новый пустой лист. Под ячейкой видно `<Figure size 640x480 with 0 Axes>`: фигура без единой рамки.

– `os.path.getsize('img/wrong.png')` — размер файла в байтах: `2398`, столько весит белая картинка. Ошибки нет, поэтому ловушка и опасна.

Ячейка 25. Сохранение фигуры по имени

```
fig.savefig('img/dtp-when.png', dpi=150, bbox_inches='tight')  # фигура из ↩
↪ прошлой ячейки
os.path.getsize('img/dtp-when.png')
```

– `fig.savefig(...)` — сохраняет фигуру `fig` из ячейки 23: переменная жива, хотя график уже показан.

– `'img/dtp-when.png'` — путь к файлу: папка `img` рядом с ноутбуком и имя файла; формат картинки задаёт окончание `.png`.

– `dpi=150` — 150 точек на дюйм, разрешение для печати.

– `bbox_inches='tight'` — граница картинки по самому рисунку: без лишних пустых полей и без обрезанных подписей.

– `os.path.getsize(...)` — `60350` байт: настоящая картинка.